ITM SKILLS UNIVERSITY

School of Future Tech

Case Study Report

on

Case Study - 181

Grand Vista — Hotel Management System

Danish Shaikh

150067251147

Index

1. Introduction to the Case Study

Grand Vista Hotel & Resorts is a terminal-based C++ application that simulates a full-featured online room booking and management system. The project was developed as Case Study 181 for the C++ Programming subject at ITM Skills University, School of Future Tech. The system enables users to browse available rooms, make bookings with automatic GST billing, checkout, and view a complete history of all bookings — all from an interactive terminal interface.

The primary aim of this case study was to demonstrate a practical, real-world application of core C++ concepts including structs, vectors, file I/O, string handling, and formatted console output. The application uses ANSI escape codes for colour-coded output and Unicode box-drawing characters to render professional-looking UI panels directly in the terminal.

Problem Background

Managing hotel bookings manually is error-prone and time-consuming. A small hotel without a dedicated POS system needs a lightweight, reliable tool to track room availability, record guest details, calculate bills with tax, and maintain a history of all transactions. No internet connectivity or external database should be required — all data must be stored locally and persist between sessions.

Abstract

This case study presents the design and implementation of **Grand Vista Hotel & Resorts**, a console-based hotel management application built with **C++17**. The system provides: a live room availability table, a guest booking flow with itemised bill preview, a GST-inclusive checkout system, and a persistent booking history — all powered by file-based storage using *rooms.txt* and *bookings.txt*. A key design improvement made during development was eliminating floating-point recalculation at checkout by storing the base amount directly in the Booking struct.

2. Problem Statement / Case Background

The hotel industry often relies on paper-based systems for small establishments, which leads to issues such as double-bookings, lost records, and incorrect billing. The challenge of this case study was to design and implement a fully-functional, zero-dependency hotel booking system that could run on any machine with a C++ compiler.

Specific constraints were:

- No external libraries or databases — all storage via plain text files.
- The system must persist data between runs (rooms, bookings, auto-incremented IDs).
- GST must be correctly computed and displayed separately from the base amount.
- The UI must be readable and professional despite being terminal-based.
- Input validation must prevent crashes from invalid user entries.

3. Problem Statement: Case Study Design

Grand Vista is architected as a single-file C++ application. All logic, UI, file I/O, and data structures reside in *hotel-management.cpp*. The system is built around the following pipeline:

Data Layer

Two structs — **Room** and **Booking** — hold all application state. These are stored in *std::vector* containers and serialised to / deserialised from tab-separated text files on program start and after every write operation.

UI Layer

All output is rendered using *cout* with ANSI escape codes (stored as *constexpr const char** constants) and Unicode box-drawing characters. A set of small helper functions (*topBar()*, *midBar()*, *botBar()*, etc.) render consistent box borders across all panels.

Logic Layer

Five core functions implement the main features: *viewRooms()*, *bookRoom()*, *checkoutRoom()*, *viewHistory()*, and the *main()* event loop with input validation.

4. Methods and Technology Used

The following technologies and C++ standard library components were used:

Component	Tool / Feature	Purpose
Language	C++17	Core language standard
Compiler	clang++ / g++	Compilation on macOS / Linux
I/O	<iostream>, <iomanip>	Console output and formatting
File Storage	<fstream>	Read/write rooms.txt, bookings.txt
Containers	std::vector	Storing Room and Booking collections
Strings	std::string, getline()	Guest name input handling
UI	ANSI escape codes	Colour-coded terminal output
Constants	constexpr const char*	Type-safe colour definitions
Limits	std::numeric_limits	Safe cin.ignore() flushing

Development Methodology

The project followed a bottom-up design approach: data structures were defined first, followed by file I/O functions, then UI helpers, and finally the high-level feature functions. This ensured that each layer depended only on layers below it, making the code easy to read and explain.

- Step 1: Define Room and Booking structs.
- Step 2: Implement loadData() / saveData() with error handling.
- Step 3: Build UI helper functions (box borders, colour constants).
- Step 4: Implement feature functions (viewRooms, bookRoom, checkoutRoom, viewHistory).
- Step 5: Wire everything in main() with validated input loop.

5. Implementation and Code Snapshots

5.1 Data Structures

The two core structs that hold all application data:

```
struct Room { int number; string type; double ratePerNight; bool available; };
```

```
struct Booking { int id; int roomNumber; string guestName; int nights; double base; double total; bool active; };
```

5.2 Application Banner and Main Menu

On launch, the system displays the hotel banner followed by the colour-coded main menu:

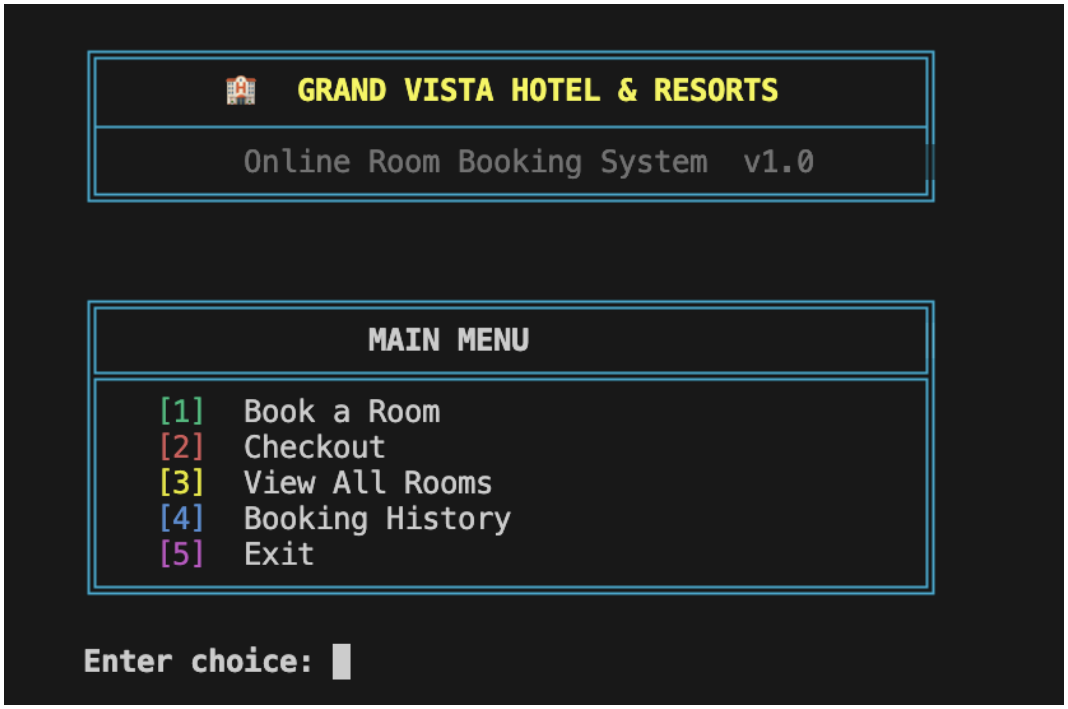

Fig 1: Application banner and main menu with colour-coded options.

5.3 Room Listing — All Available

Option [3] displays all 7 rooms in a formatted table. Available rooms are highlighted in green and occupied rooms in red:

Enter choice: 1

Room	Type	Rate/Night	Status
101	Single	Rs.1500	Available
102	Single	Rs.1500	Available
103	Double	Rs.2500	Available
104	Double	Rs.2500	Available
201	Deluxe	Rs.3500	Available
202	Suite	Rs.5000	Available
301	Penthouse	Rs.10000	Available

Enter room number:

Fig 2: Room listing table showing all rooms available on fresh start.

5.4 Booking Flow and Bill Preview

Option [1] prompts the user for a room number, guest name, and number of nights. A booking preview box immediately shows the base amount, 12% GST, and total:

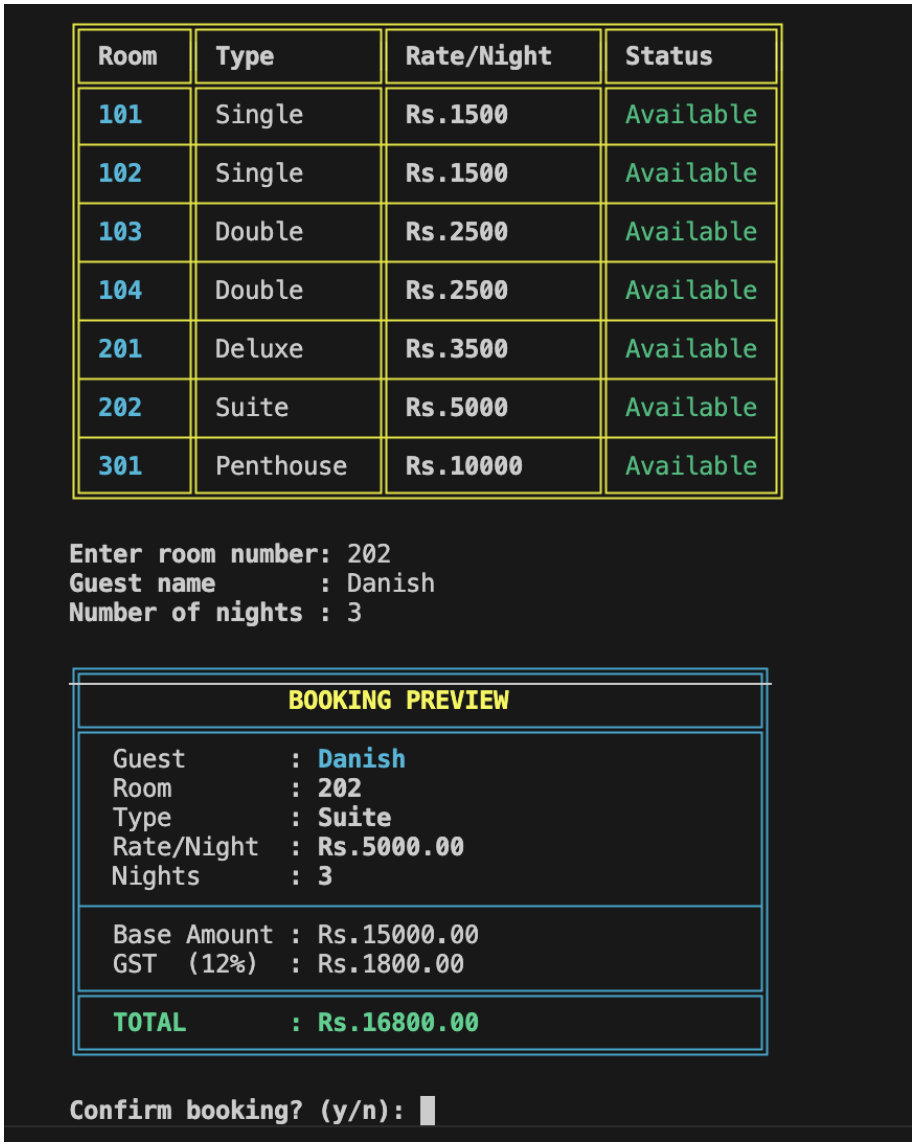

Fig 3: Booking form with live bill preview showing GST breakdown.

5.5 Booking Confirmation

After confirming with 'y', the booking is saved and a success message shows the Booking ID:

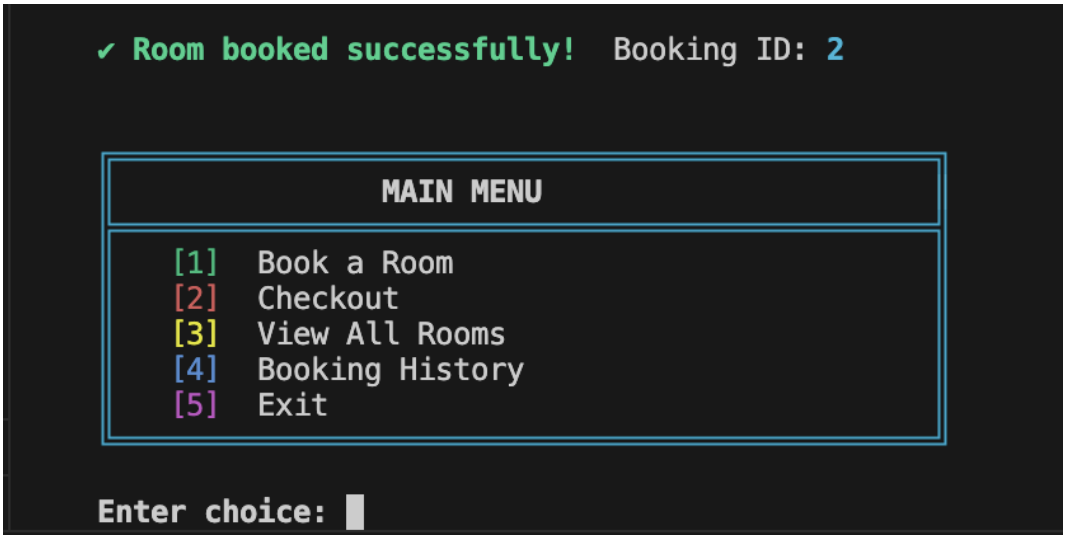

Fig 4: Booking confirmation with auto-assigned Booking ID.

5.6 Room Status Update

After booking Room 202, the room table correctly reflects the 'Occupied' status in red:

Enter choice: 3

Room	Type	Rate/Night	Status
101	Single	Rs.1500	Available
102	Single	Rs.1500	Available
103	Double	Rs.2500	Available
104	Double	Rs.2500	Available
201	Deluxe	Rs.3500	Available
202	Suite	Rs.5000	Occupied
301	Penthouse	Rs.10000	Available

Fig 5: Room 202 (Suite) marked as Occupied after booking.

5.7 Booking History

Option [4] shows all bookings. Active bookings appear in green; completed ones are dimmed:

Enter choice: 4

BOOKING HISTORY

#2 Room 202 Danish 3 nts Rs. 16800.00 Active

Fig 6: Booking History panel showing active booking #2.

5.8 Checkout and Final Bill

Option [2] lists active bookings, lets the user select by ID, shows the final bill (base amount and GST read directly from stored values — no recalculation), and confirms checkout:

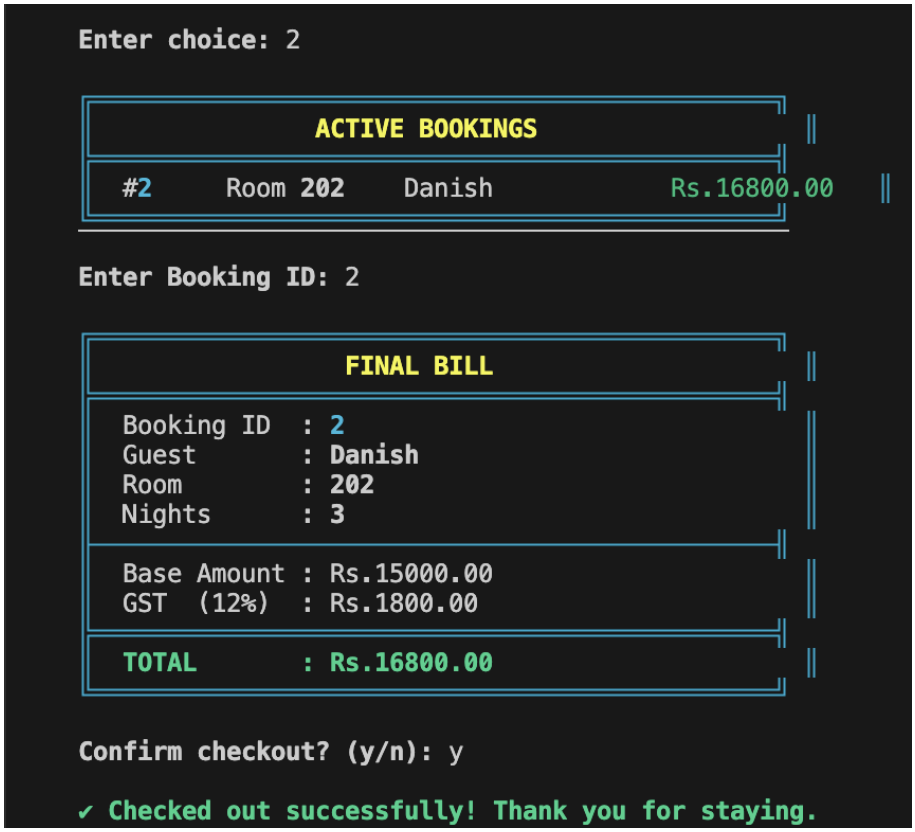

Fig 7: Checkout flow with final bill and checkout success message.

5.9 History After Checkout

Post-checkout, the booking history updates the status from 'Active' to 'Done':

Fig 8: Booking History showing completed booking marked as Done.

6. Results / Conclusion

Key Outcomes

- All 5 features (view rooms, book, checkout, history, exit) implemented and fully functional.
- File persistence verified — bookings survive program restarts.
- GST correctly calculated at 12% and displayed as a separate line item.
- Input validation catches non-integer menu entries, non-existent rooms, and invalid night counts.
- Code compiles with zero warnings under clang++ -std=c++17 -Wall -Wextra.

Code Quality Improvements Made

During review and refactoring, the following improvements were applied to meet C++ best practices:

- Replaced all #define macro colour constants with constexpr const char* for type safety.
- Replaced 'using namespace std' with explicit using declarations for each std:: symbol.
- Added is_open() error checks in saveData() before writing to both files.
- Replaced cin.ignore(1000, '\n') with the correct std::numeric_limits approach.
- Stored base amount directly in the Booking struct to avoid floating-point recalculation at checkout.
- Removed the unused roomIcon() function that was defined but never called.

Conclusion

Grand Vista Hotel & Resorts successfully demonstrates the application of core C++ programming concepts in a practical, real-world context. The project achieves all stated objectives: it is fully functional, produces accurate billing, persists data between sessions, handles invalid input gracefully, and presents a professional user interface despite being a terminal application.

The case study also served as an exercise in code quality — each refactoring decision (removing macros, removing dead code, adding error handling) was driven by identifiable C++ best practices, making the codebase easy to read and explain in a viva examination setting.

7. References

1. Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.

2. cppreference.com — std::vector, std::fstream, std::numeric_limits documentation.

3. ANSI Escape Codes Reference — *https://en.wikipedia.org/wiki/ANSI_escape_code*

4. ITM Skills University — C++ Programming Course Notes, School of Future Tech, 2025.

5. GitHub Repository — *https://github.com/Danish1323/Grand-Vista*